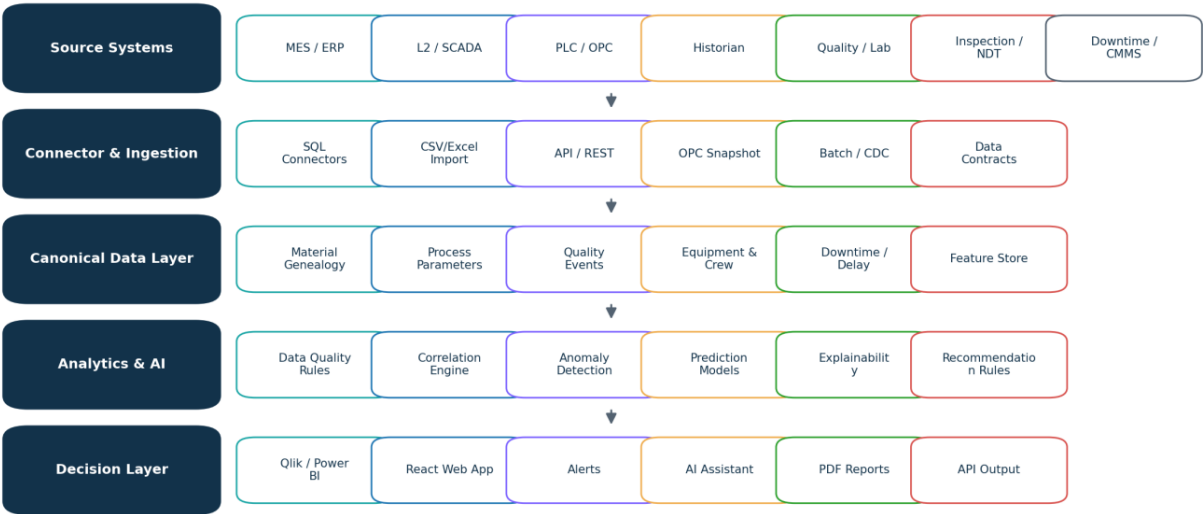


PlantProcess IQ

Technical Blueprint for a Generic Manufacturing Quality Intelligence & Prediction Platform

PlantProcess IQ - Generic Reference Architecture

A domain-neutral intelligence layer above MES, L2, historian, inspection, quality and ERP data.



Prepared for: Karim Gamal Elsayed
Version 1.0 | 2026

1. Executive Technical Summary

PlantProcess IQ is designed as a generic intelligence layer above existing MES, Level 2, historian, quality, laboratory, inspection, downtime and ERP systems. It does not replace MES, SCADA or BI. It connects process and production data end-to-end, builds material genealogy, identifies correlations between process parameters and quality outcomes, predicts quality risk, and explains suspected contributors in a dashboard and AI-assisted report.

Core design principle

Build a domain-neutral manufacturing data model first. Industry-specific wording such as heat, coil, batch, tire, roll, lot, paper reel or aluminum billet must be configured as metadata, not hard-coded logic.

2. Product Objectives

- Connect fragmented production, process, inspection and quality data across the plant.
- Normalize data into a canonical manufacturing model that works for process and discrete manufacturing.
- Build genealogy from raw material, batch or order through process steps to final product and quality decision.
- Detect missing, duplicated, delayed or inconsistent production/quality records.
- Run statistical correlation and machine-learning models to identify risk contributors.
- Predict quality risk before the next process step and provide explainable recommendations.
- Offer dashboards in Qlik/Power BI for early MVP, and a React-based web application for a scalable product.

3. Recommended Product Architecture

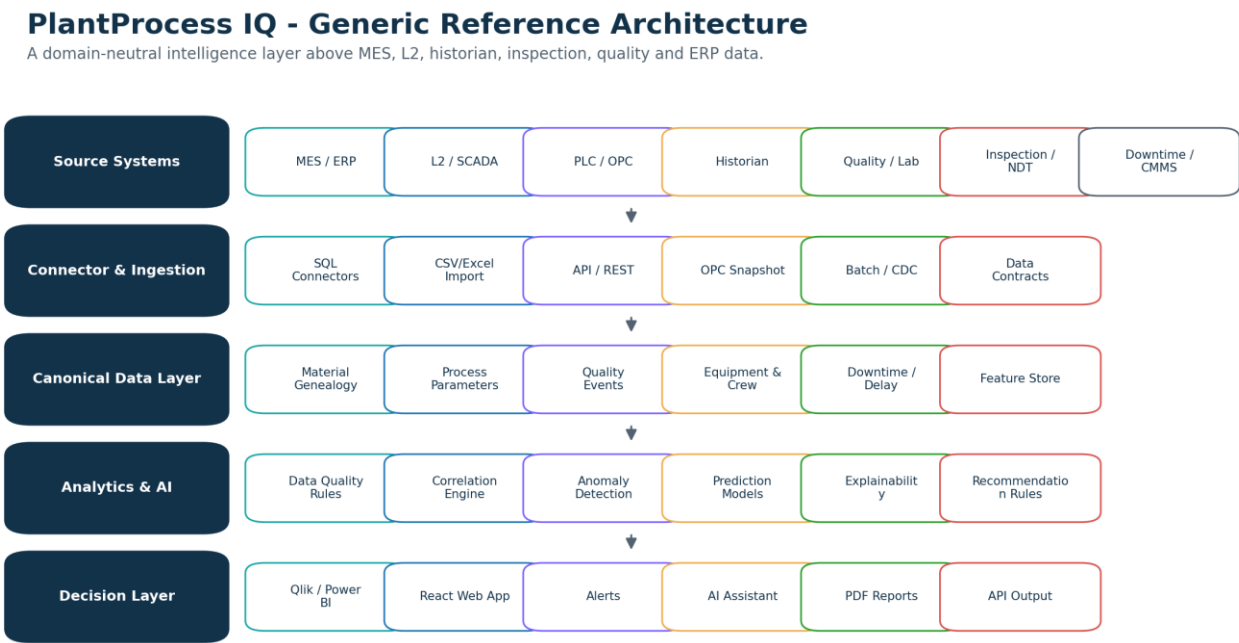


Figure 1 - Reference architecture for a generic manufacturing quality intelligence platform.

The architecture should separate the product into layers. This avoids building a tool that only works for one plant or one industry. Each layer has a clear responsibility and can be replaced or extended later.

Layer	Purpose	Recommended Technology
Data connector layer	Read from MES, L2, PLC/OPC exports,	C#/.NET connectors, Python ingestion

	historian, lab, inspection, downtime, ERP and BI data sources.	scripts, ODBC/JDBC, REST APIs, CSV/Excel import, OPC UA snapshots where permitted.
Staging layer	Store raw snapshots exactly as received for auditability and repeatable testing.	PostgreSQL or SQL Server; object storage/parquet later for large data.
Canonical data layer	Transform plant-specific source data into generic entities: material, operation, equipment, parameter, quality event and genealogy.	PostgreSQL/SQL Server relational model with JSONB/JSON columns for flexible attributes.
Analytics layer	Run data quality rules, correlation, anomaly detection and feature engineering.	Python/scikit-learn, SQL analytics, optional ML.NET for .NET-native models.
Prediction service	Serve trained models and calculate material risk scores.	Python FastAPI or .NET API; ONNX Runtime for framework-neutral inference later.
Decision layer	Dashboards, AI explanations, reports, recommendations and APIs.	React + .NET API for product; Qlik/Power BI for pilot dashboards.

4. Database Strategy

4.1 Recommended MVP database

For the first version, use PostgreSQL as the product database because it is open, strong for relational modeling, supports declarative table partitioning, and provides JSON/JSONB types for flexible attributes. PostgreSQL partitioning is useful for large time-based observation tables, and JSONB is useful for plant-specific parameters while the canonical model is still evolving.

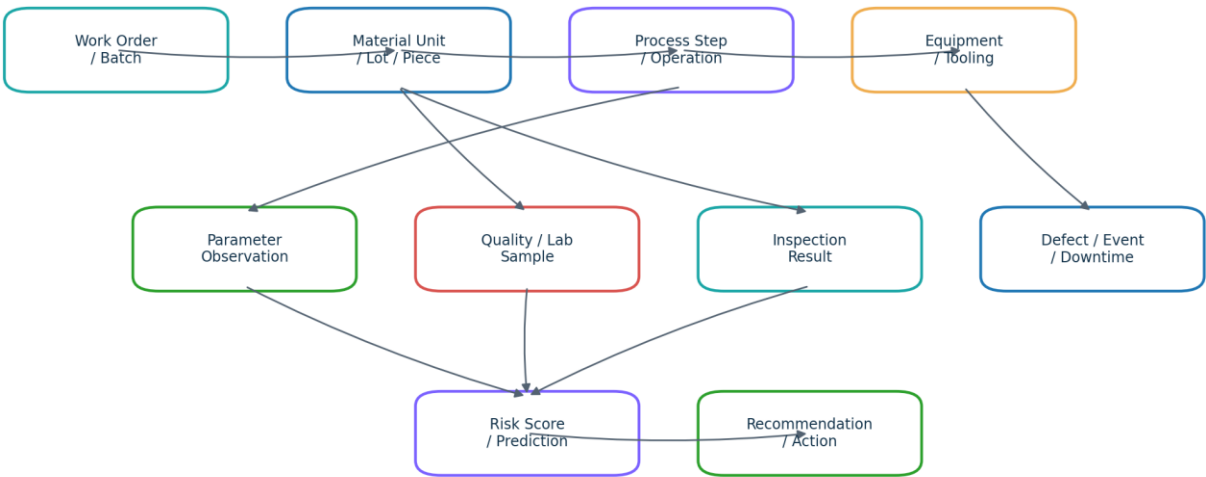
4.2 Enterprise customer database support

- Support Microsoft SQL Server and Oracle as source systems because many MES/L2 environments already use them.
- Keep source connectors separate from the product storage layer.
- Never force a plant to change its existing MES/L2 databases.
- For pilots, accept database exports if direct connection is politically or technically difficult.
- For production, prefer read-only database users, API exports, or scheduled extracts approved by the customer IT/security team.

4.3 Core database schema

Generic Manufacturing Canonical Model

Designed for steel, pharma, tire, paper, automotive, aluminum and other process/discrete industries.



Key design principle: store industry-specific names as configurable metadata; keep the core model stable.

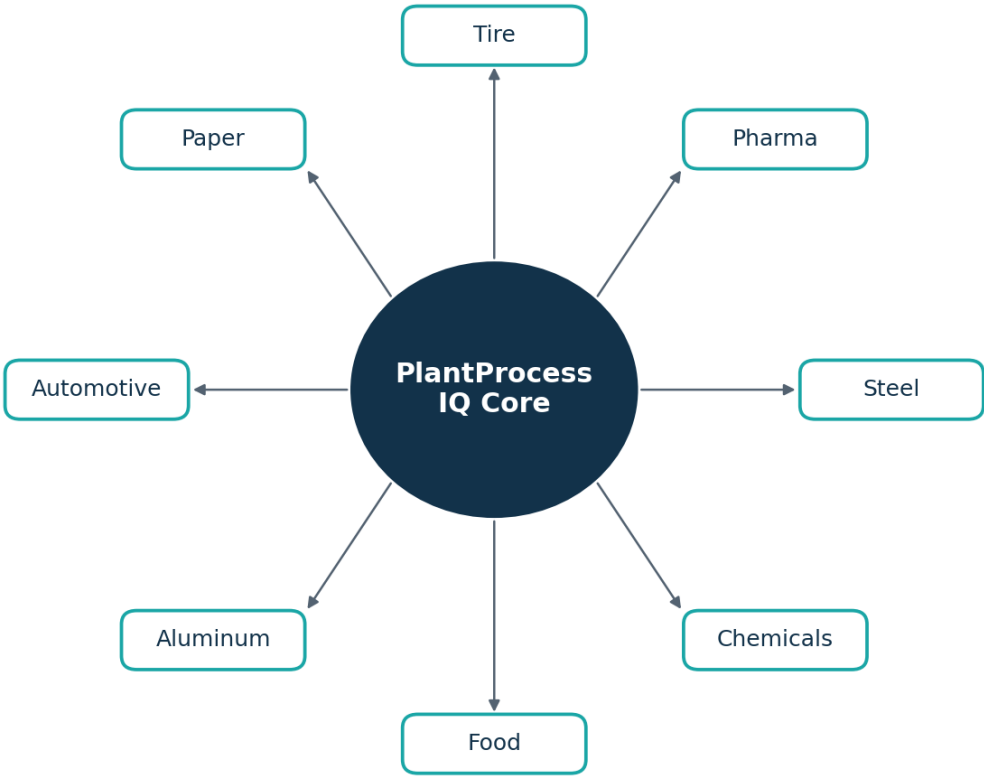
Figure 2 - Generic canonical model that supports multiple manufacturing domains.

Entity	Generic Meaning	Steel Example	Other Industry Example
Plant	Manufacturing site	Flat steel plant	Pharma factory / paper mill
Area / Line	Production area or process line	Caster, HSM, Cold Mill	Filling line, tire curing, paper machine
Work order / Batch	Planned production unit	Heat/order	Batch, lot, work order
Material unit	Traceable physical material	Slab, coil, rail, section	Tablet batch, tire, roll, car part
Operation	Manufacturing step	LF treatment, casting, rolling	Mixing, curing, coating, assembly
Parameter observation	Measured process value	Casting speed, temperature	Pressure, humidity, torque
Quality event	Defect/result/status	Crack, inclusion, flatness	Contamination, blister, dimension defect
Equipment context	Machine/tool/asset identity	Mould, tundish, stand	Mold, press, reactor, cutter
Crew/shift context	Operational context	Shift A / crew ID	Operator group / line team
Prediction	Risk model output	Surface defect risk	Reject probability / quality class

5. Generic Data Model Strategy

To make the platform generic across steel, pharma, tire, paper, automotive and aluminum, the product should use a graph-like genealogy model on top of a relational database. The central concept is not “coil” or “heat”; the central concept is a traceable material or batch unit that moves through operations and accumulates parameters, events and quality outcomes.

One Generic Core - Many Manufacturing Domains



Reusable entities: batch / material / operation / equipment / parameter / quality event / prediction / recommendation

Figure 3 - Domain-neutral core with industry-specific configuration packs.

Generic Concept	Configuration Examples
Material unit types	Heat, slab, coil, rail, batch, tire, paper roll, aluminum billet, car component
Operation types	Melting, casting, rolling, coating, mixing, curing, assembly, inspection, packaging
Parameter catalog	Temperature, pressure, speed, force, chemistry, humidity, composition, delay duration
Quality event catalog	Crack, inclusion, center buckle, contamination, blister, dimension defect, surface defect
Equipment catalog	Furnace, ladle, mould, mill stand, reactor, press, cutter, inspection device
Domain rules	Steel grade rules, pharma batch release rules, tire curing rules, paper moisture rules

6. Data Ingestion and Integration

- Start with offline data import for MVP: CSV, Excel, SQL export, and synthetic datasets.
- Add read-only SQL connectors for pilot: SQL Server, Oracle, PostgreSQL.
- Add scheduled extraction jobs for production: nightly batch, shift-based load, or near-real-time incremental load.

- Add optional OPC/historian integration only when the customer has a secure approved method and clear data ownership.
- Use a source-system registry to track where each parameter came from and when it was loaded.
- Store raw data snapshots before transformations to make analysis reproducible.

Integration Mode	Use When	Risk Level	Recommendation
CSV/Excel import	First demo or customer data readiness assessment	Low	Best first step. Fast and safe.
Direct SQL read-only	Pilot with IT approval	Medium	Use read-only credentials and limit queries.
API/REST extraction	Modern MES/QMS/ERP systems	Medium	Good for controlled access.
CDC/replication	Large production deployment	High	Only after strong customer IT involvement.
OPC/historian stream	Near-real-time risk scoring	High	Later phase, not first MVP.

7. Backend and Service Design

Use a modular monolith first. Avoid microservices too early. The product needs fast iteration, a clean domain model and clear module boundaries more than distributed complexity. Later, the ML service, connector service and reporting service can be separated.

Module	Responsibility	Suggested Stack
Identity & tenant module	Users, roles, plant/site isolation, permissions	.NET API + PostgreSQL/SQL Server
Connector module	Source registrations, import jobs, mapping definitions	.NET workers + Python scripts where useful
Canonical model module	Material genealogy, operations, parameters and quality events	.NET domain model + EF Core
Analytics module	Data-quality checks, correlations, statistics	Python + SQL + background jobs
ML module	Training, validation, model registry, prediction	Python/scikit-learn first; ML.NET/ONNX for serving later
Dashboard API module	Expose KPIs, risk scores, genealogy and correlation outputs	ASP.NET Core Web API
Report module	PDF/Excel export and investigation reports	.NET document generation + templates

8. Frontend and Dashboard Strategy

Do not build everything in Qlik Sense only. Qlik is excellent for fast BI dashboards and first customer pilots, but the core product should remain independent. Build the data intelligence engine and expose it through APIs. Then offer multiple visualization options.

Option	Best Use	Advantages	Limitations
Qlik Sense dashboard	Fast pilot, business users, strong visual demo	Very fast dashboard delivery; good associative exploration; can connect to external analytic engines via SSE.	Not ideal as the full product core or complex workflow application.
Power BI dashboard	Customers already using Microsoft BI	Good adoption in many companies; strong enterprise reporting.	Can become report-heavy without product workflow.
React web application	Full product experience and workflows	Full control: genealogy search, risk review, approvals, AI explanations, reports.	More development effort than BI tools.
Hybrid: React + embedded BI	Best long-term strategy	Product controls the workflow; BI tools handle rich dashboards.	Needs clean API and licensing strategy.

Quality Risk Dashboard



Figure 4 - Example dashboard mockup for quality risk and parameter correlation.

9. Qlik, Python, R, C# and React: Recommended Decision

Recommended stack decision

Use C#/.NET + React for the product platform, Python for machine learning, PostgreSQL/SQL Server for storage, and Qlik Sense/Power BI as optional dashboard layers. Do not make Qlik the core product database or workflow engine.

Capability	Recommended Tool	Why
Core backend	.NET 10 LTS or current enterprise LTS	Matches Karim's C# strength and enterprise industrial environments.
Frontend product	React + TypeScript	Good for interactive product UI, genealogy search, risk review and admin tools.
Pilot dashboards	Qlik Sense first, Power BI optional	Fast visual delivery and customer-friendly analytics.
ML training	Python / scikit-learn	Strong ecosystem for correlation, preprocessing, classification and explainable models.
.NET-native ML	ML.NET	Useful when models need to run inside .NET environments or edge/server applications.
Model serving	FastAPI or .NET API; ONNX Runtime later	Keeps models accessible to the product and can support multiple frameworks.
Data storage	PostgreSQL for MVP; SQL Server for Windows-heavy customers	PostgreSQL is flexible; SQL Server may fit enterprise customers using Microsoft stack.

Qlik Sense can call external calculation engines through analytic connections / server-side extensions. This makes Qlik useful for advanced analytics pilots where Python or R performs calculations and Qlik presents the results. However, the product should keep the AI logic in a separate service so that the same intelligence can be consumed by Qlik, Power BI, React UI, APIs, and reports.

10. Analytics and Machine Learning Design

Machine Learning Lifecycle

Start with interpretable correlation and defect-risk scoring before moving to advanced AI.

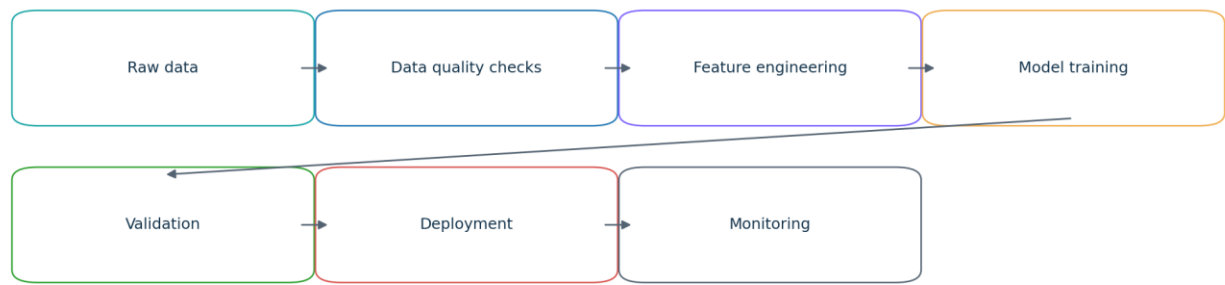


Figure 5 - ML lifecycle: start with correlation and explainability before advanced prediction.

Stage	Method	Purpose	Output
Data profiling	Missing values, duplicates, timestamp gaps, distribution checks	Know if the customer data is usable.	Data readiness report.
Correlation analysis	Pearson/Spearman, chi-square, group defect rates, parameter bins	Identify candidate relationships between parameters and quality outcomes.	Correlation matrix and risk contributors.
Feature engineering	Rolling windows, lag features, process-route aggregation, grade/equipment encoding	Convert raw process data into model-ready features.	Feature table per material/batch.
Baseline prediction	Logistic regression, random forest, gradient boosting	Predict probability of defect/rejection/high-risk quality event.	Risk score + feature importance.
Anomaly detection	Isolation Forest, robust z-score, clustering	Detect unusual process patterns even without labeled defects.	Anomaly score and abnormal cases.
Explainability	Feature importance, SHAP later, rule-based explanation templates	Make model output acceptable to process engineers.	Reason summary per material.
Recommendation	Rule engine + process windows + expert validation	Suggest action without pretending the model is process authority.	Recommended investigation/action list.

Important technical rule: the product should distinguish correlation from root cause. The dashboard should say “suspected contributor” or “risk indicator” unless process experts validate causality.

11. Security, Legal and Deployment Requirements

- For the MVP, use synthetic data only.
- For customer pilots, sign NDA and use customer-approved sample exports or read-only access.
- Never collect personal data such as operator names unless legally approved; prefer crew/shift codes and anonymized IDs.
- Prefer on-prem deployment for sensitive manufacturing customers.
- Add role-based access control, audit logs, encryption at rest/in transit, and clear data-retention rules.

- Create data-processing agreements and clarify ownership of trained models and derived insights.
- Avoid any use of SMS/client confidential data, internal documents, code or screenshots.

Deployment Options

The product should work where plant data is allowed to live.



Figure 6 - Deployment options from demo to enterprise rollout.

12. MVP Implementation Steps

1. Define one generic use case: process-to-quality correlation for one route and one defect family.
2. Design canonical schema: material, operation, parameter observation, quality event, equipment, crew/shift, downtime and prediction.
3. Build synthetic data generator with 100,000+ rows across multiple process steps.
4. Implement PostgreSQL/SQL Server schema and import pipeline.
5. Build ETL mapping layer from synthetic source tables to canonical model.
6. Create data quality rules for missing IDs, duplicate events, missing timestamps and broken genealogy.
7. Build correlation engine and first feature-importance report.
8. Train baseline defect-risk model and validate with synthetic hidden patterns.
9. Create dashboard in Qlik Sense or Power BI first, and later React UI for product workflow.
10. Prepare customer pilot package: one-page offer, sample report, architecture diagram and demo dashboard.

13. References

- Microsoft .NET support policy and release lifecycle: <https://dotnet.microsoft.com/en-us/platform/support/policy>
- ML.NET documentation: <https://learn.microsoft.com/en-us/dotnet/machine-learning/>
- scikit-learn documentation: <https://scikit-learn.org/>
- Qlik Sense analytic connections / Server-Side Extension: https://help.qlik.com/en-US/sense-developer/November2025/Content/Sense_Helpsites/AnalyticConnections.htm
- PostgreSQL declarative partitioning: <https://www.postgresql.org/docs/current/ddl-partitioning.html>
- PostgreSQL JSON/JSONB types: <https://www.postgresql.org/docs/current/datatype-json.html>
- ONNX Runtime documentation: <https://onnxruntime.ai/docs/>